



Ingeniería de Software

Sistemas de Bases de Datos Distribuidas

MsC. Fernando Estrella

**PROYECTO FINAL - Sistema de
Integración y Transformación de
Datos: E-Commerce**

Marvin Zambrano

Martes, 28 de julio de 2026

TABLA DE CONTENIDO

1.	INTRODUCCIÓN	2
2.	DESCRIPCIÓN DEL CASO DE USO	2
3.	MODELO DE DATOS	3
4.	ARQUITECTURA DEL SISTEMA	4
5.	REGLAS DE TRANSFORMACIÓN	6
6.	IMPLEMENTACIÓN	8
7.	RESULTADOS Y PRUEBAS	12
8.	CONCLUSIONES Y TRABAJO A FUTURO	16

1. INTRODUCCIÓN

En el avance tecnológico actual, las organizaciones modernas enfrentan desafíos crecientes en la gestión de datos distribuidos. Las arquitecturas empresariales ya no dependen de un solo motor de base de datos, sino que adoptan enfoques híbridos y en los casos de mayor demanda un ecosistema en equilibrio. El presente proyecto resuelve el problema de la duplicación y consistencia de datos mediante el desarrollo de una arquitectura distribuida que combina bases de datos relacionales (PostgreSQL) para datos estructurados y bases de datos NoSQL (MongoDB) para información flexible y escalable. La justificación de este sistema radica en la necesidad de sincronizar bidireccionalmente el inventario transaccional con el catálogo dinámico de una tienda en línea, evitando conflictos de versión y garantizando la integridad referencial.

2. DESCRIPCIÓN DEL CASO DE USO

Se ha seleccionado como caso de uso una Plataforma de Comercio Electrónico multicanal.

Se implementa gestión relacional con el motor de base de datos PostgreSQL el cual se encarga de gestionar de manera estricta las transacciones, el inventario físico y los pedidos de los usuarios. Adicional a ello la gestión documental se encargará del almacenamiento de productos, el cual necesita una estructura dinámica para manejar variaciones como tallas, colores, reseñas de clientes y metadatos adicionales sin restricciones de los esquemas rígidos.

Consiguiendo que el sistema mantenga una sincronización en tiempo real o en base a la demanda entre el inventario transaccional de la base de datos

relacional (PostgreSQL) y el catalogo de productos en la base de datos no relacional (MongoDB), asegurando que las compras se reflejen inmediatamente en la reducción del stock en ambos motores.

3. MODELO DE DATOS

El diseño de los datos del presente proyecto tiene el enfoque de persistencia entre diferentes motores donde cada motor de base de datos maneja una cara de trabajo en la que se desempeña mejor. Por un lado el modelo relacional con PostgreSQL esta estructurado con la normalización de datos e integridad referencial. Las entidades transaccionales como los usuarios, pedidos y el stock deben cumplir ACID para evitar las anomalías en las ventas sobre el inventario o registros que no tengan relación, esto se logra con el uso de claves foráneas y borrados en cascada.

La llave de sincronización o el enlace que permite la comunicación entre estos dos ecosistemas es el tratamiento del código unitario conocido como SKU, el cual tendrá el rol de clave primaria en la tabla stock en el modelo relacional y en modelo no relacional este SKU cumple la función de identificador único del documento, este procedimiento permite una sincronización bidireccional, es decir, que permite a los servicios ubicar el registro en ambos motores en tiempo real, mapear las transformaciones de datos de SQL a JSON y viceversa, con la finalidad de garantizar la consistencia de información. La Ilustración 1 muestra el modelo de datos del proyecto.

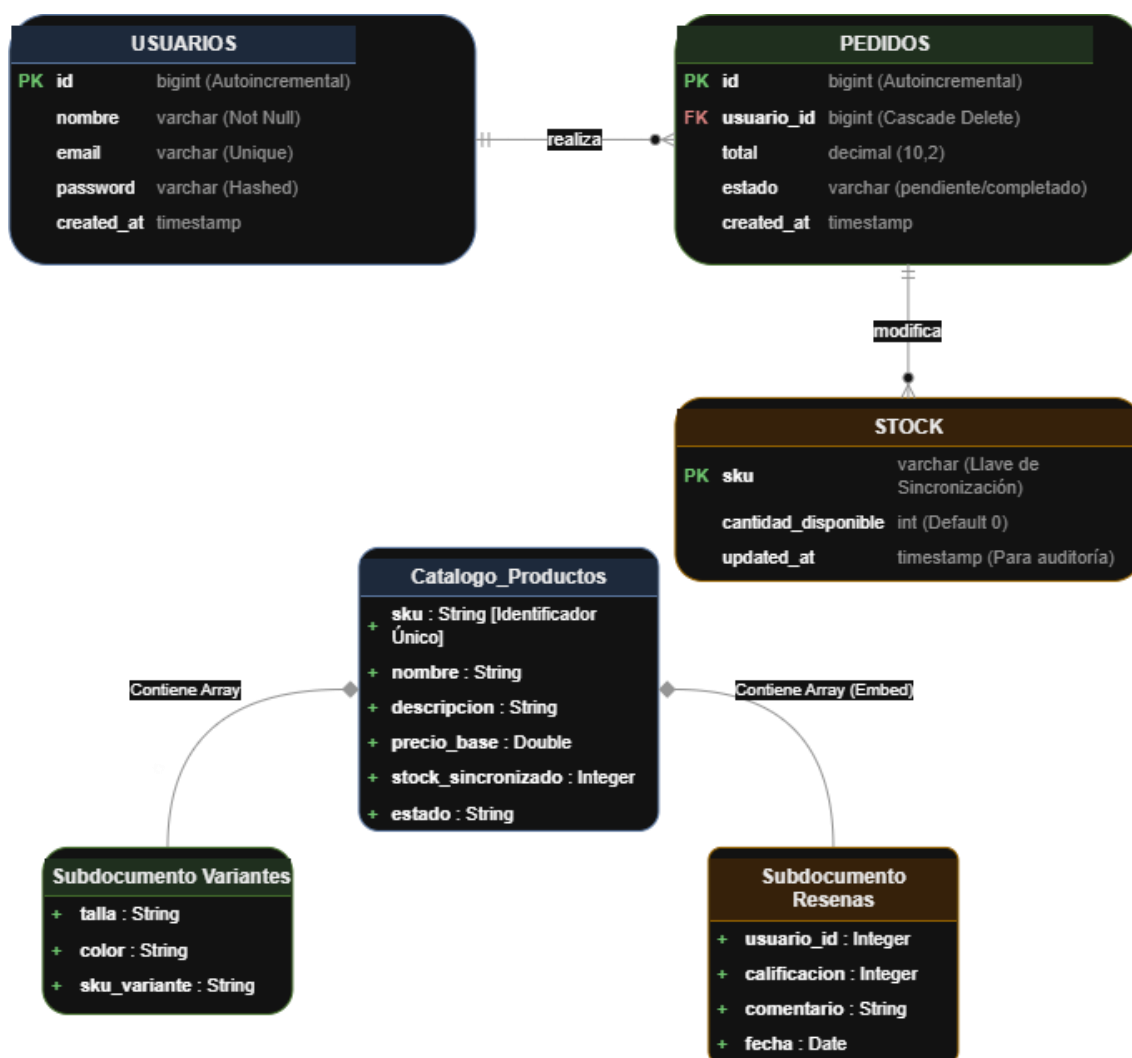


Ilustración 1 Modelo de Datos SQL y NoSQL

4. ARQUITECTURA DEL SISTEMA

El sistema se diseñó bajo una arquitectura distribuida orientada a servicios SOA simplificada, con un patrón Cliente-Servidor de múltiples capas. Esta decisión permite desacoplar la lógica de presentación de la lógica de negocio y persistencia de datos para que el sistema pueda escalar independientemente y tenga tolerancia a fallos.

4.1 Desglose de Capas

Capa de Presentación: Se construyó como una SPA con React.js empaquetado con Vite, esta capa se encarga de gestionar el estado de la

interfaz de usuario en tiempo real, sin necesidad de actualizaciones. Para los diseños se implementó Tailwind CSS para un diseño responsive y adaptativo. La comunicación con el servidor se realiza mediante peticiones HTTP con Axios, la que entrega los datos recibidos en formatos JSON.

Capa de Enrutamiento (Backend Controller): Esta construido sobre el framework de Laravel (PHP), en donde se exponen endpoints RESTful con los métodos GET, POST, PUT, DELETE quienes actuaran como comunicadores ente la capa de presentación. Además, de políticas CORS para autorizar únicamente peticiones al frontend, que permita la validación de datos en cada transacción para evitar inyección de datos.

Capa de Lógica de Negocio: Se diseño una clase dedicada que permite al backend acceder a cada uno de los motores. Esta capa centraliza la regla principal que es la de transformación de datos bidireccional, la cual permite orquestar transacciones complejas, como lo hace, lee el estado en un motor, mapea los tipos de datos que contiene y realiza la escritura en los que corresponde, además en el caso de falla existen registros de fallas con Logs de Laravel en las operaciones importantes de la plataforma, que permita crear una pista de auditoría.

Capa de Persistencia: Esta capa utiliza el patrón Active Record a trabas de Eloquent para la abstracción de las consultas SQL. Para la base de datos relacional PostgreSQL se usa el drive *pgsql* nativo de PDO, que aseguras las transacciones ACID para las operaciones de ventas y pedidos. En cambio, para la no relacional MongoDB, el drive que utiliza permite la manipilación de documentos BSON sin tener que escribir comandos de

agregación adicionales, lo que facilita el mapeo de los datos dentro del documento.

4.2 Resiliencia y Manejo de Errores

La arquitectura asume que, en un entorno de bases de datos distribuidas, las fallas de red son inevitables. Por ello, el flujo de sincronización está protegido por bloques *try-catch*. Si el motor transaccional PostgreSQL procesa una venta, pero el motor documental MongoDB se encuentra offline, la excepción es capturada, el usuario no experimenta una caída del sistema, y el error se registra en el log interno para una posterior conciliación manual o reintento.

5. REGLAS DE TRANSFORMACIÓN

Para asegurar la consistencia y la integridad de los datos entre dos paradigmas de almacenamiento distintos el relacional y el documental, el sistema implementa un conjunto de reglas de transformación estrictas gestionadas por la capa de sincronización.

Regla 1: Transformación SQL a JSON (Sincronización de Inventario)

Esta regla asegura que las operaciones transaccionales afecten al catálogo de productos en tiempo real, garantizando que el usuario final no vea productos agotados. El evento desencadenante sucede cuando una operación de UPDATE en la tabla stock de PostgreSQL.

Extracción de Datos: El orquestador extrae la llave primaria SKU y el valor actualizado de la columna *quantity_available* de la base de datos relacional.

Lógica de Mapeo y Transformación:

- Se aplica una regla de conservación de tipo donde el motor de PHP asegura que el valor numérico extraído de la consulta SQL no sea convertido accidentalmente a un tipo cadena.
- Se mapea la columna tabular *quantity_available* hacia la llave JSON "stock_sincronizado".

Aplicación de la Regla: El sistema localiza el documento BSON en MongoDB mediante el identificador SKU y ejecuta una actualización parcial. Se modifica únicamente el nodo del inventario, preservando intactos los arreglos anidados como variantes o reseña que coexisten en el mismo documento.

Regla 2: Transformación JSON a SQL (Creación y Aplanamiento de Datos)

Esta regla resuelve el problema de "aplanar" un documento complejo para que pueda ser consumido por un motor relacional estricto.

Evento Desencadenante: Se inserta un nuevo documento JSON en la colección *producto_catalog* de MongoDB por ejemplo el lanzamiento de un nuevo producto desde el panel de administración.

Extracción de Datos: El sistema obtiene el payload completo del documento, el cual contiene múltiples nodos y subdocumentos.

Lógica de Mapeo y Transformación:

- Filtrado Selectivo: El servicio descarta intencionalmente toda la metadata flexible que PostgreSQL no necesita ni soporta de forma nativa nodos de arrays, descripciones largas, calificaciones.

- Traducción de Esquema: Extrae exclusivamente las llaves principales del nivel raíz del JSON: "sku" y "stock_inicial".
- Mapea la llave JSON "sku" a la columna relacional sku para garantizar su unicidad.
- Mapea la llave JSON "stock_inicial" a la columna relacional *stock_quantity*.

Aplicación de la Regla: Se genera y ejecuta una sentencia INSERT o UPSERT para evitar colisiones en PostgreSQL. Esta transformación convierte exitosamente un documento no estructurado en una fila transaccional válida, permitiendo que el producto esté disponible inmediatamente para la venta bajo control ACID.

Regla 3: Propagación de Borrado (Eliminación Distribuida)

Para evitar la existencia de registros huérfanos, la plataforma implementa una regla de eliminación simétrica.

Regla: Cuando se emite una instrucción DELETE desde la interfaz, el sistema no distingue el motor de origen. Utilizando el SKU como nexos, la capa de servicios traduce la petición en dos comandos paralelos: un borrado de fila (Row Delete) en PostgreSQL y un borrado de documento (Document Delete) en MongoDB. Si uno de los dos motores falla, el bloque de manejo de excepciones registra en los logs de auditoría para su revisión, manteniendo la integridad referencial.

6. IMPLEMENTACIÓN

El desarrollo del backend se realizó sobre el framework Laravel basado en el lenguaje de programación PHP, implementando el patrón de dependencias para

desacoplar los controladores de la lógica de sincronización. A continuación, se detallan los componentes fundamentales del código fuente que operativizan la persistencia y el flujo del E-Commerce.

6.1 Configuración de Conexiones Distribuidas

El núcleo de la arquitectura requirió habilitar múltiples manejadores de bases de datos simultáneos. En el archivo de configuración central (config/database.php), se definieron las credenciales y los drivers tanto para el motor relacional nativo PostgreSQL como para el motor documental MongoDB, integrado a través del paquete en Laravel. La Ilustración 2 detalla las conexiones realizadas para cada motor dentro del proyecto.

```
'pgsql' => [
    'driver' => 'pgsql',
    'url' => env('DB_URL'),
    'host' => env('DB_HOST', '127.0.0.1'),
    'port' => env('DB_PORT', '5432'),
    'database' => env('DB_DATABASE', 'ecommerce'),
    'username' => env('DB_USERNAME', 'DEV1'),
    'password' => env('DB_PASSWORD', '*****'),
    'charset' => env('DB_CHARSET', 'utf8'),
    'prefix' => '',
    'prefix_indexes' => true,
    'search_path' => 'public',
    'sslmode' => env('DB_SSLMODE', 'prefer'),
],

'mongodb' => [
    'driver' => 'mongodb',
    'dsn' => env('MONGODB_URI', 'mongodb://localhost:27017'),
    'database' => env('MONGODB_DATABASE', 'tienda_catalogo'),
],
```

Ilustración 2 Conexiones de Motores de Base de Datos

6.2 Servicio de Sincronización – Reglas de Transformación

Se implementó una clase de servicio dedicada exclusivamente a aplicar las reglas de mapeo de datos entre los dos paradigmas SQL ↔ JSON. Este

componente asegura que cualquier cambio de inventario en una base de datos se traduzca al tipo de dato correcto en la otra, manejando las excepciones internamente para no interrumpir el flujo de la aplicación. La Ilustración 3 muestra la funcionalidad de sincronización y cumpliendo una de las reglas de transformación de JSON a SQL.

```
/**
 * Sincroniza un nuevo producto desde MongoDB hacia PostgreSQL.
 * Cumple con la regla de transformación JSON -> SQL.
 */
public function syncMongoToSql($sku)
{
    try {
        $productoMongo = ProductCatalog::where('sku', $sku)->first();

        if ($productoMongo) {
            $inventario = Stock::updateOrCreate(
                ['sku' => $productoMongo->sku],
                ['cantidad_disponible' => $productoMongo->stock_sincronizado ?? 0]
            );

            Log::info("Sincronización exitosa [MongoDB -> PostgreSQL] | SKU: {$sku}");

            return true;
        }
        return false;
    } catch (\Exception $e) {
        Log::error("Error en sincronización inversa SKU {$sku}: " . $e->getMessage());
        return false;
    }
}
```

Ilustración 3 Método de Sincronización de Mongo a PostgreSQL

6.3 Orquestación de la API – Controlador RESTful

El Controlador actúa como la interfaz de comunicación con el frontend que esta implementado con React. El cual expone métodos que combinan la lectura de ambos motores en tiempo real para el monitoreo del desajuste o sincronía y maneja las operaciones de escritura (CRUD). El método *index* que se muestra en la Ilustración 4 indica como el controlador obtiene los datos del catalogo de productos en MongoDB y busca el stock en la base de datos relacional PostgreSQL para responder el catálogo de productos.

```

/**
 * Obtiene el catálogo de MongoDB y el inventario de PostgreSQL
 * para el Dashboard comparativo.
 */
public function index()
{
    $catalogoMongo = ProductCatalog::all();
    $inventariosSql = Stock::all()->keyBy('sku');

    $datosCombinados = $catalogoMongo->map(function ($producto) use ($inventariosSql) {
        $sqlItem = $inventariosSql->get($producto->sku);

        return [
            'sku' => $producto->sku,
            'nombre' => $producto->nombre,
            'stock_mongo' => $producto->stock_sincronizado,
            'stock_postgres' => $sqlItem ? $sqlItem->quantity_available : 'No existe',
            'estado_sincronizacion' => ($sqlItem && $sqlItem->quantity_available === $producto->stock_sincronizado)
        ];
    });

    return response()->json($datosCombinados);
}

```

Ilustración 4 Controlador para Catálogo de productos

6.4 Interfaz de Usuario y Consumo Asíncrono

En la capa de presentación (Frontend React), se implementaron peticiones HTTP asíncronas utilizando Axios. La interfaz consume los endpoints unificados y reacciona de forma reactiva al estado de la base de datos, proporcionando retroalimentación visual inmediata al usuario tras operaciones críticas como ventas, abastecimiento o eliminación.

En la

```

const simularCompra = async (sku, stockActual) => {
    if (stockActual <= 0) {
        Swal.fire('Sin Stock', 'No hay unidades disponibles para vender.', 'warning');
        return;
    }
    try {
        const nuevoStock = stockActual - 1;
        await axios.put(`${API_URL}/inventario/${sku}`, { cantidad: nuevoStock });
        Swal.fire({
            toast: true,
            position: 'top-end',
            icon: 'success',
            title: 'Venta registrada',
            showConfirmButton: false,
            timer: 2000
        });
        cargarDatos();
    } catch (error) {
        Swal.fire('Error', 'Fallo al procesar la venta.', 'error');
    }
};

```

Ilustración 5 se muestra el proceso de reducción del inventario cuando se realiza una venta unitaria, con las condiciones de que el stock debe estar sincronizado entre ambos, que exista stock del producto.

```
const simularCompra = async (sku, stockActual) => {
  if (stockActual <= 0) {
    Swal.fire('Sin Stock', 'No hay unidades disponibles para vender.', 'warning');
    return;
  }
  try {
    const nuevoStock = stockActual - 1;
    await axios.put(`${API_URL}/inventario/${sku}`, { cantidad: nuevoStock });
    Swal.fire({
      toast: true,
      position: 'top-end',
      icon: 'success',
      title: 'Venta registrada',
      showConfirmButton: false,
      timer: 2000
    });
    cargarDatos();
  } catch (error) {
    Swal.fire('Error', 'Fallo al procesar la venta.', 'error');
  }
};
```

Ilustración 5 Simulación de venta en el sistema

7. RESULTADOS Y PRUEBAS

Para validar la correcta integración de la arquitectura distribuida y el cumplimiento de los requisitos funcionales (RF-001 a RF-018), se llevaron a cabo pruebas de integración sobre los flujos de trabajo principales.

A continuación, se documenta la evidencia del comportamiento del software en tiempo real:

Prueba 1: Visualización Consolidada del Estado de Datos (Lectura)

Se comprobó la capacidad del backend para consultar paralelamente la tabla de inventario en PostgreSQL y la colección de productos en MongoDB, consolidando la información mediante la API REST y desplegándola en la interfaz cliente (React).

Gestión de Bases de Datos Distribuidas (PostgreSQL + MongoDB) Refrescar Tablero

Crear Nuevo Producto

+ Sincronizar Producto en Motores de BDD

SKU	PRODUCTO	STOCK (MONGO)	STOCK (POSTGRES)	ESTADO SINC.	
TSHIRT-LARAVEL-01	Camiseta Laravel Oficial	149	149	Sincronizado	Abastecer
MUG-LARAVEL-02	Taza de Cerámica Laravel	29	29	Sincronizado	Abastecer
CAP1	Manta	10	10	Sincronizado	Abastecer

Ilustración 6 Visualización de Estados

Prueba 2: Creación de Producto y Sincronización Inversa (JSON → SQL)

Se ejecutó la prueba de envío desde el formulario del frontend creando un nuevo producto con datos como SKU, Nombre, Precio y Stock Inicial.

Resultado: El documento BSON completo debe crearse en MongoDB, y paralelamente debe crearse una nueva tupla en PostgreSQL conteniendo únicamente el sku y la cantidad_disponible.

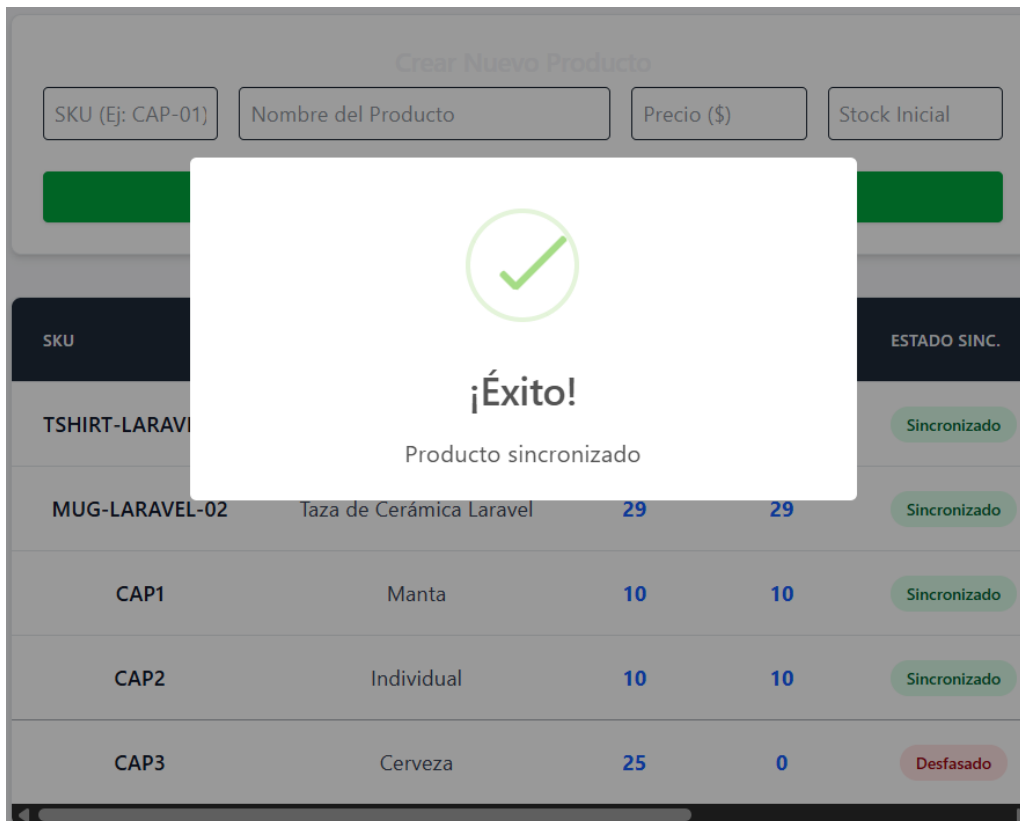


Ilustración 7 Sincronización de Items

Prueba 3: Simulación de Transacciones E-Commerce (SQL → JSON)

Se realizaron pruebas de estrés funcional simulando ventas con la reducción de stock en 1 unidad y abastecimiento incremento de stock en 10 unidades sobre registros existentes.

Resultado: La modificación efectuada sobre la base de datos relacional debe actualizar automáticamente el campo `stock_sincronizado` en el documento NoSQL correspondiente sin romper los arreglos variantes, reseñas.

PRODUCTO	STOCK (MONGO)	STOCK (POSTGRES)	ESTADO SYNC.	ACCIONES		
Camiseta Laravel Oficial	149	149	Sincronizado	Abastecer	Vender	Eliminar
Taza de Cerámica Laravel	29	29	Sincronizado	Abastecer	Vender	Eliminar
Manta	9	9	Sincronizado	Abastecer	Vender	Eliminar
Individual	30	30	Sincronizado	Abastecer	Vender	Eliminar
Cerveza	19	19	Sincronizado	Abastecer	Vender	Eliminar

Ilustración 8 Transacciones de Productos Inicial

PRODUCTO	STOCK (MONGO)	STOCK (POSTGRES)	ESTADO SYNC.	ACCIONES		
Camiseta Laravel Oficial	147	147	Sincronizado	Abastecer	Vender	Eliminar
Taza de Cerámica Laravel	28	28	Sincronizado	Abastecer	Vender	Eliminar
Manta	9	9	Sincronizado	Abastecer	Vender	Eliminar
Individual	25	25	Sincronizado	Abastecer	Vender	Eliminar
Cerveza	18	18	Sincronizado	Abastecer	Vender	Eliminar

Ilustración 9 Transacciones de productos Final

Prueba 4: Trazabilidad y Logs de Auditoría (RF-014)

Se revisó el archivo de almacenamiento de logs de la aplicación (storage/logs/laravel.log) para asegurar que el controlador deje constancia de cada evento de sincronización o fallo interceptado.

```
1417 [2026-07-27 02:29:10] local.INFO: Sincronización exitosa [MongoDB -> PostgreSQL] | SKU: CAP3
1418 [2026-07-27 02:30:07] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP3 | Nuevo Stock: 10
1419 [2026-07-27 02:30:09] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP3 | Nuevo Stock: 10
1420 [2026-07-27 02:30:20] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 20
1421 [2026-07-27 02:30:36] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP3 | Nuevo Stock: 20
1422 [2026-07-27 02:30:39] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 30
1423 [2026-07-27 02:30:42] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP1 | Nuevo Stock: 9
1424 [2026-07-27 02:30:44] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP3 | Nuevo Stock: 19
1425 [2026-07-27 02:30:49] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 29
```



```

1430 [2026-07-27 02:32:55] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: TSHIRT-LARAVEL-01 | Nuevo Stock: 148
1431 [2026-07-27 02:33:02] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: MUG-LARAVEL-02 | Nuevo Stock: 28
1432 [2026-07-27 02:33:05] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: TSHIRT-LARAVEL-01 | Nuevo Stock: 147
1433 [2026-07-27 02:33:09] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 28
1434 [2026-07-27 02:33:13] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP3 | Nuevo Stock: 18
1435 [2026-07-27 02:33:17] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 27
1436 [2026-07-27 02:33:20] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 26
1437 [2026-07-27 02:33:23] local.INFO: Sincronización exitosa [PostgreSQL -> MongoDB] | SKU: CAP2 | Nuevo Stock: 25

```

Ilustración 10 Logs de Sincronización de Transacciones

8. CONCLUSIONES Y TRABAJO A FUTURO

8.1 Conclusiones

Viabilidad de la Persistencia: Se demostró de manera práctica que el uso combinado de motores relacionales como PostgreSQL y no relacionales como MongoDB resuelve eficientemente las limitaciones de los sistemas monolíticos tradicionales, permitiendo manejar datos estructurados con rigor ACID y datos no estructurados con alta flexibilidad.

Efectividad de interconexión mediante SKU: La elección estratégica del identificador único de producto (SKU) como cadena de texto en ambos motores eliminó la dependencia de identificadores, facilitando el mapeo directo y la sincronización bidireccional sin introducir sobrecostos de procesamiento.

Desacoplamiento Mediante Servicios: La implementación del patrón de capa de servicios dentro de Laravel garantizó que las reglas de transformación SQL ↔ JSON se mantuvieran aisladas de los controladores, permitiendo un código mantenible, escalable y fácil de probar.

8.2 Trabajo a Futuro

Procesamiento Asíncrono vía Colas (Queues): Para entornos de alta concurrencia o tráfico masivo, se propone reemplazar la sincronización síncrona actual por un sistema de colas utilizando Redis. Esto permitirá que

las compras se procesen instantáneamente en PostgreSQL y que las actualizaciones a MongoDB se ejecuten en segundo plano sin afectar la velocidad de respuesta del cliente.

Manejo de Transacciones (Conciliación Distribuida): Implementar un patrón para garantizar la consistencia eventual estricta mediante mecanismos de compensación en caso de fallas prolongadas de red o caídas temporales de uno de los motores de base de datos.

Implementación de Webhooks: Exponer webhooks para notificar a sistemas externos de logística o inventario de terceros cuando el stock de un producto llegue a niveles críticos menores o iguales a cero.

9. REFERENCIAS

Laravel Core Team. (2026). *Laravel Documentarion*. Obtenido de Laravel Documentation: Database & Eloquent ORM.: <https://laravel.com/docs>

Martin, F. (2012). *MartinFowler.com*. Obtenido de Polyglot Persistence. MartinFowler.com.: <https://martinfowler.com/bliki/PolyglotPersistence.html>

MongoDB Inc. (2026). *MongoDB*. Obtenido de MongoDB Laravel Integration Package Manual.: <https://www.mongodb.com/docs/drivers/php/laravel-mongodb/>

PostgreSQL Global Development Group. (2026). *PostgreSQL 16 Documentation: Transactions and ACID Properties*. Obtenido de PostgreSQL: <https://www.postgresql.org/docs/>

10. PRESENTACIÓN

El siguiente enlace contiene la presentación del proyecto.

- Parte Teórica:

<https://drive.google.com/file/d/1T5QdxybWhRMiiEhxCtv7rOONTOp8tKHf/view?usp=sharing>

- Parte Práctica:

<https://drive.google.com/file/d/10n16zuD9RgGJORLXRpaD1QvfPO5unW8D/view?usp=sharing>

- Repositorio:

https://github.com/marvinzambrano310/Proyecto_Final_BDD